

Mission complète — Migration de Digitalecomland vers Supabase

Tu travailles sur le dépôt GitHub :
[solver492/Digitalecomland](https://github.com/solver492/Digitalecomland)

1. Objectif général

Transformer l'application Digitalecomland afin qu'elle fonctionne réellement avec Supabase comme backend de données et système d'authentification, tout en conservant l'interface et les fonctionnalités existantes.

L'application doit fonctionner selon cette architecture :

text



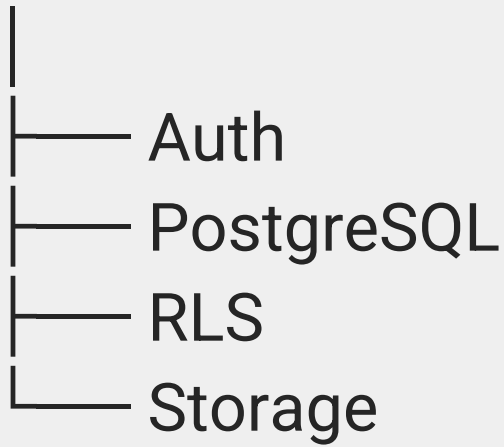
Frontend React/Vite



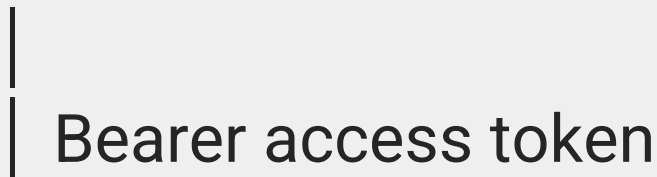
Supabase Auth



Supabase



Frontend



Backend Express / API



Supabase PostgreSQL

Le projet doit rester compatible avec un déploiement sur Hostinger.

Ne pas utiliser Vercel.

Ne pas dépendre de Replit.

2. IMPORTANT – commencer par analyser le dépôt

Avant de modifier quoi que ce soit :

1. Analyse toute l'architecture du dépôt.
2. Analyse artifacts/digital-ecom-land.
3. Analyse artifacts/api-server.
4. Analyse lib/api-client-react.
5. Analyse lib/api-spec.
6. Analyse lib/api-zod.
7. Analyse lib/db.
8. Analyse tous les fichiers de configuration :
 - package.json
 - pnpm-workspace.yaml
 - vite.config.ts
 - configurations TypeScript
 - variables d'environnement
 - configuration du serveur
 - scripts de build

scripts de démarrage.

9. Identifie toutes les fonctionnalités utilisant actuellement PostgreSQL/Drizzle.
10. Identifie toutes les fonctionnalités dépendant de Replit.
11. Identifie toutes les routes API existantes.
12. Identifie toutes les pages frontend.
13. Identifie le système d'authentification actuellement présent ou absent.
14. Identifie les endroits où l'application suppose qu'un utilisateur est connecté.
15. Identifie les endroits où les données utilisateur doivent être protégées.

Ne réécris pas inutilement l'application.

Le but est de faire évoluer l'architecture existante, pas de refaire le frontend.

3. Projet Supabase à utiliser

Le projet Supabase existant est :

text



Project ID:
nfoefhwmngjatbqyibclp

L'application doit utiliser les variables
d'environnement suivantes :

env



VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=

Règle de sécurité absolue

SUPABASE_SERVICE_ROLE_KEY doit être

utilisée UNIQUEMENT côté serveur.

Elle ne doit :

- jamais apparaître dans le frontend ;

- jamais être préfixée par VITE_;

- jamais être envoyée au navigateur ;

- jamais être commitée dans GitHub ;

- jamais apparaître dans les logs.

Le frontend doit uniquement utiliser :

```
env
```



```
VITE_SUPABASE_URL
```

```
VITE_SUPABASE_PUBLISHABLE_KEY
```

4. Supabase — schéma déjà existant

Le projet Supabase possède déjà ces tables :

text



profiles

suppliers

products

product_media

orders

withdrawals

telegram_messages

social_publications

Ne recrée pas ces tables si elles existent déjà.
Avant toute migration supplémentaire, compare
le schéma existant avec le code du dépôt.

5. Authentication Supabase

Implémenter une authentification complète avec Supabase Auth.

Utiliser de préférence :

text



```
@supabase/supabase-js
```

si son ajout est compatible avec l'architecture actuelle.

Créer un client Supabase frontend centralisé, par exemple :

text



```
artifacts/digital-ecom-land/src/lib/  
supabase.ts
```

ou un emplacement équivalent adapté à l'architecture existante.

Le client doit utiliser :

ts



```
createClient(  
  import.meta.env.VITE_SUPABASE_URL,  
  
  import.meta.env.VITE_SUPABASE_PUBLIC_KEY  
)
```

6. Authentification utilisateur

Implémenter :

inscription ;

connexion ;

déconnexion ;

récupération de session ;

renouvellement automatique de session ;

détection de session existante ;
récupération de l'utilisateur courant ;
protection des pages privées.

Les fonctionnalités doivent utiliser Supabase Auth et non un système d'authentification inventé séparément.

7. Profil utilisateur

Lorsqu'un utilisateur Supabase est créé :

text



auth.users



public.profiles

Le profil correspondant doit être créé automatiquement.

Le profil possède notamment :

text



id

full_name

phone

email

city

brand_name

bank_name

rib_number

payment_method

role

created_at

updated_at

Le champ :

text



role

doit supporter au minimum :

text



affiliate
admin

Le [profiles.id](#) doit correspondre au :

text



auth.users.id

8. Gestion du token côté frontend

Le projet possède déjà un système dans :

text



lib/api-client-react

permettant de configurer un AuthTokenGetter.
Connecter ce système à Supabase.

Le principe doit être :

text



Supabase session



access_token



api-client-react



Authorization: Bearer <access_token>



Express API

Le frontend ne doit pas avoir besoin de gérer manuellement les JWT.

Utiliser :

ts



```
supabase.auth.getSession()
```

ou le mécanisme officiel approprié pour récupérer le token courant.

9. Backend Express — authentication

Le backend actuel utilise Express.

Ajouter un middleware d'authentification.

Exemple conceptuel :

text



```
Authorization: Bearer  
<SUPABASE_ACCESS_TOKEN>
```

Le middleware doit :

1. vérifier la présence du header Authorization ;
2. récupérer le Bearer token ;
3. vérifier le token auprès de Supabase ;
4. récupérer l'utilisateur Supabase ;
5. attacher l'utilisateur au contexte de la requête ;
6. refuser les requêtes non authentifiées sur les routes protégées.

Ne jamais simplement décoder un JWT sans vérifier sa signature.

Utiliser les mécanismes officiels Supabase adaptés au backend.

10. Séparer les routes publiques et privées

Identifier toutes les routes API.

Créer une séparation claire :

text



PUBLIC

- health
- autres routes réellement publiques

AUTHENTICATED

- profile
- dashboard
- products
- orders
- withdrawals
- autres données privées

ADMIN

- gestion utilisateurs
- gestion fournisseurs
- gestion produits
- gestion publications
- administration

Ne pas protéger /healthz avec l'authentification.

—

11. Autorisation par rôle

Ne pas considérer qu'un utilisateur authentifié est automatiquement administrateur.

Récupérer son profil :

text



public.profiles.role

et vérifier :

text



affiliate
admin

Créer un middleware ou helper du type :

text



requireAuth
requireAdmin

Un utilisateur affiliate ne doit jamais pouvoir exécuter une route réservée à admin.

12. Protection contre l'accès aux données d'un autre utilisateur

Auditer toutes les routes existantes.

Chercher notamment :

text



profile

orders

withdrawals

products

dashboard

wallet

Une requête utilisateur ne doit jamais pouvoir fournir un ID arbitraire pour récupérer ou modifier les données privées d'un autre utilisateur.

Utiliser systématiquement l'identité issue du token Supabase :

text



auth.uid()

ou l'identité vérifiée côté serveur.

Ne jamais faire confiance à :

text



req.body.userId

req.query.userId

req.params.userId

pour déterminer l'identité de l'utilisateur connecté.

13. Base de données — Drizzle

Le dépôt possède actuellement :

text



lib/db

avec Drizzle/PostgreSQL.

Analyser précisément son utilisation.

Ne pas supprimer Drizzle automatiquement.

Déterminer pour chaque table si elle doit être :

Option A

Accédée directement via Supabase.

ou

Option B

Accédée uniquement via l'API Express.

L'architecture finale doit éviter deux systèmes concurrents qui écrivent les mêmes données sans raison.

14. Architecture recommandée

Pour les données sensibles et les opérations

métier :

text



Frontend



Express API



Supabase PostgreSQL

Pour l'authentification :

text



Frontend



Supabase Auth

Pour les données qui peuvent être consultées
directement depuis le frontend et qui sont

correctement protégées :

text



Frontend



Supabase

Ne jamais exposer la service-role key.

15. Produits

La table :

text



products

doit être intégrée au fonctionnement réel de

l'application.

Vérifier :

affichage des produits ;

catégories ;

prix d'achat ;

prix de vente conseillé ;

frais de livraison ;

fournisseur ;

image ;

statut actif ;

date de création ;

source Telegram.

Les données doivent provenir de Supabase/API et non de données mockées lorsque les données réelles sont disponibles.

16. Médias produits

Utiliser :

text



product_media

pour gérer plusieurs images/médias associés à un produit.

Préparer l'architecture pour Supabase Storage.

Ne pas stocker de grosses images directement en base PostgreSQL.

Le flux doit pouvoir devenir :

text



Telegram



Media



Supabase Storage



product_media



Product

17. Fournisseurs

Intégrer :

text



suppliers

dans l'architecture.

Les produits doivent pouvoir être associés à un fournisseur via :

text



products.supplier_id

Vérifier les routes/API correspondantes.

18. Telegram

Préparer l'application pour le futur flux :

text



Telegram fournisseur



Telethon / n8n / automation



telegram_messages



products



product_media



CRM Digitalecomland

La table :

text



telegram_messages

doit pouvoir conserver :

fournisseur ;

channel ID ;

message ID ;

texte ;

date ;

payload original ;

statut processed.

Ne pas implémenter une connexion Telegram fictive.

Préparer uniquement les API/services nécessaires au pipeline réel.

19. Commandes

Intégrer :

text



orders

avec le système existant.

Vérifier :

création ;

consultation ;

statut ;

produit ;
client ;
adresse ;
prix ;
frais ;
bénéfice ;
dates.

Les utilisateurs ne doivent voir que les commandes auxquelles ils ont réellement accès.

20. Retraits / Wallet

Intégrer :

text



withdrawals

au système d'authentification.

Une demande de retrait doit être associée au compte connecté.

Vérifier toutes les routes existantes liées à :

text



wallet

withdrawals

bank_name

rib_number

amount

status

Ne jamais permettre à un affiliate de modifier ou consulter arbitrairement le retrait d'un autre compte.

21. Publications sociales

Intégrer :

text



social_publications

dans l'architecture.

Préparer les champs :

text



product_id

platform

destination

content

status

external_post_id

scheduled_at

published_at

error_message

L'application doit être prête pour le futur système :

text



Product



Social publication



Facebook / Instagram / autres
plateformes

Ne pas implémenter de fausse publication si aucune API réelle n'est connectée.

22. RLS Supabase

Conserver RLS activé.

Le principe de sécurité doit être :

text



auth.uid()

pour déterminer l'utilisateur.

Les profils ont déjà leurs politiques RLS.

Pour les autres tables, ne pas créer une politique dangereuse du genre :

```
sql
```



```
USING (true)
```

juste pour faire disparaître les warnings.

Avant de créer une politique, déterminer le propriétaire réel de la donnée.

23. Problème important du schéma actuel

Les tables métier suivantes ne possèdent pas toutes actuellement de colonne propriétaire

explicite :

text



products

orders

withdrawals

social_publications

telegram_messages

suppliers

product_media

Avant d'ajouter des politiques RLS utilisateur sur ces tables, analyser le modèle métier.

Si une colonne est nécessaire, proposer une migration propre, par exemple :

text



user_id uuid references auth.users(id)

uniquement lorsque cela correspond réellement au modèle métier.

Ne pas ajouter user_id partout sans analyser les relations.

24. Dashboard

Le dashboard doit fonctionner avec le véritable utilisateur Supabase.

À l'ouverture :

text



session



user



profile



dashboard

Si aucun utilisateur n'est connecté :

text



redirect → login

Ne pas utiliser de faux utilisateur local.

25. Protection frontend

Créer un mécanisme clair du type :

text



ProtectedRoute

ou équivalent.

Les pages privées doivent être protégées.

Exemple :

text



/dashboard

/orders

/wallet

/profile

Les pages publiques doivent rester accessibles.

26. Gestion de session

Utiliser les événements Supabase Auth afin que l'application réagisse correctement à :

text



SIGNED_IN

SIGNED_OUT

TOKEN_REFRESHED

USER_UPDATED

Éviter les états incohérents entre :

text



React

Supabase Auth

API Express

27. Variables d'environnement

Mettre à jour les fichiers .env.example si nécessaire.

Exemple :

env



VITE_SUPABASE_URL=

VITE_SUPABASE_PUBLISHABLE_KEY=

SUPABASE_URL=

SUPABASE_SERVICE_ROLE_KEY=

Ne jamais mettre de vraies clés secrètes dans Git.

Vérifier également :

text



.gitignore

et ajouter les fichiers .env* sensibles si nécessaire.

Conserver éventuellement .env.example.

28. Supprimer les dépendances Replit

Rechercher dans tout le dépôt :

text



replit

REPLIT

REPL_ID

REPLIT_DB

replit.dev

replit.com

Identifier chaque occurrence.

Pour chaque occurrence :

1. déterminer son utilité ;
2. remplacer par une solution compatible avec Hostinger/Supabase ;
3. supprimer les dépendances inutiles ;
4. ne pas casser le fonctionnement existant.

Ne pas supprimer aveuglément une dépendance sans comprendre son rôle.

29. Vercel

Le projet doit être destiné à Hostinger.

Rechercher :

text



vercel

VERCEL

et supprimer/remplacer les éléments devenus inutiles.

Ne pas introduire de nouvelle dépendance Vercel.

30. Hostinger

Le résultat final doit pouvoir être déployé sur un serveur Hostinger.

Préserver une architecture :

text



Node.js

Express

React/Vite

Supabase

Le build frontend doit être reproductible.

Le backend doit pouvoir démarrer avec une commande standard.

Vérifier :

text



pnpm install

pnpm build

pnpm start

ou les commandes équivalentes réellement définies par le dépôt.

Ne pas introduire une architecture dépendante d'un serveur spécifique à Replit.

31. Configuration API

Le frontend utilise actuellement un proxy Vite vers :

text



/api

Vérifier que cette architecture reste correcte en développement.

En production, utiliser la configuration appropriée pour servir :

text



frontend

+

API Express

sur Hostinger.

32. Gestion des erreurs

Ajouter une gestion propre des erreurs :

text



401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Les erreurs Supabase ne doivent pas exposer :

service role key ;
stack traces sensibles ;
credentials ;
secrets ;
informations internes PostgreSQL.

33. TypeScript

Conserver TypeScript strict autant que possible.
Après modification :

text



```
pnpm install  
pnpm build
```

puis :

text



pnpm typecheck

si le script existe.

Corriger toutes les erreurs introduites par la migration.

Ne pas masquer les erreurs avec :

ts



as any

sauf justification réelle.

34. Tests fonctionnels minimum

Vérifier au minimum :

Auth

text



Inscription

Connexion

Session persistante

Déconnexion

Profil

text



Création automatique

Lecture

Modification

API

text



Requête sans token → 401

Token valide → accès

Utilisateur non-admin → 403 sur route admin

Produits

text



Lecture

Création/modification selon rôle

Commandes

text



Création

Lecture

Protection utilisateur

Wallet

text



Création withdrawal

Lecture de ses propres withdrawals

35. Ne pas casser l'interface

L'objectif n'est pas de refaire le design.

Conserver :

- composants ;

- pages ;

- navigation ;

- styles ;

- responsive ;

- UX ;

- logique métier existante.

Modifier uniquement ce qui est nécessaire pour connecter réellement l'application à Supabase.

36. Supprimer les mocks progressivement

Rechercher :

text



mock

dummy

fake

sample

demo

hardcoded

localStorage

Identifier ce qui représente de vraies données

métier.

Remplacer progressivement les mocks par :

text



Supabase/API

Mais ne pas supprimer les données de démonstration utilisées uniquement pour le design sans vérifier leur rôle.

37. localStorage

Auditer l'utilisation de :

text



localStorage
sessionStorage

Ne pas utiliser localStorage comme système d'authentification parallèle.

La session doit être gérée par Supabase Auth. Si une donnée métier est persistée localement uniquement pour contourner la base de données, remplacer ce mécanisme par l'API/Supabase.

38. Client API

Le système existant :

text



lib/api-client-react

doit rester utilisable.

Connecter son :

text



```
setAuthTokenGetter(...)
```

à Supabase.

Le résultat attendu :

text



```
apiClient
```



```
get current Supabase access token
```



```
Authorization Bearer
```



```
Express
```

39. Backend — ne pas faire

confiance au frontend

Toutes les autorisations doivent être vérifiées côté serveur.

Même si le frontend cache un bouton admin, le backend doit quand même refuser :

text



affiliate → admin endpoint

Le frontend n'est jamais une couche de sécurité suffisante.

40. Sécurité Supabase

Vérifier :

RLS ;

permissions ;

fonctions SECURITY DEFINER ;

policies ;

accès anon ;

accès authenticated ;

accès service_role.

Les fonctions SECURITY DEFINER ne doivent pas être inutilement exécutables publiquement. Ne pas exposer de fonction permettant de contourner RLS.

41. Ne pas modifier les secrets

Ne jamais demander à l'utilisateur de mettre une :

text



SUPABASE_SERVICE_ROLE_KEY

dans le frontend.

Ne jamais commit une vraie clé.

Utiliser uniquement les variables d'environnement.

42. Migration de base de données

Si une modification SQL est nécessaire :

1. créer une migration explicite ;
2. ne pas modifier directement la production sans migration ;
3. documenter les changements ;
4. vérifier les contraintes existantes ;
5. vérifier les foreign keys ;

6. vérifier RLS ;
 7. vérifier les indexes.
-

43. Compatibilité avec le schéma existant

Avant chaque changement de schéma :

text



inspect existing schema



compare with Drizzle schema



compare with API types



compare with frontend usage



migration

Ne pas supprimer une colonne existante simplement parce qu'elle n'est pas utilisée actuellement.

44. Documentation

Créer ou mettre à jour :

```
text
```



```
README.md
```

avec :

Installation

```
bash
```



```
pnpm install
```

Variables d'environnement

Expliquer :

text



VITE_SUPABASE_URL

VITE_SUPABASE_PUBLISHABLE_KEY

SUPABASE_URL

SUPABASE_SERVICE_ROLE_KEY

Développement

Expliquer comment démarrer frontend + API.

Build

Expliquer comment produire la version production.

Supabase

Hostinger

Expliquer les variables d'environnement nécessaires au serveur.

45. Vérification finale

Avant de considérer la mission terminée :

Code

text



- [] TypeScript compile
- [] Build réussi
- [] Pas d'erreur ESLint critique
- [] Pas de secret dans Git
- [] Pas de service-role key frontend
- [] Pas de dépendance Replit inutile
- [] Pas de dépendance Vercel inutile

Auth

text



- [] Signup
- [] Login
- [] Logout
- [] Session
- [] Token refresh
- [] Protected routes

Backend

text



- [] requireAuth
- [] requireAdmin
- [] Authorization Bearer
- [] 401
- [] 403

Supabase

text



- [] profiles
- [] products
- [] product_media
- [] suppliers
- [] orders
- [] withdrawals
- [] telegram_messages
- [] social_publications

Sécurité

text



- [] RLS conservé
- [] policies vérifiées
- [] SECURITY DEFINER sécurisé
- [] pas de données cross-user
- [] service role uniquement serveur

46. Règle importante concernant les modifications

Ne fais pas une réécriture complète du projet.

Travaille par étapes :

text



1. Audit
2. Auth Supabase
3. Token frontend → API
4. Auth middleware Express
5. Autorisation
6. Connexion données
7. Suppression dépendances Replit
8. Compatibilité Hostinger
9. Tests
10. Build final

Après chaque étape importante, vérifie que le projet compile.

47. Git

Créer des commits propres et descriptifs.

Exemples :

text



feat: integrate Supabase authentication

feat: add Express Supabase auth
middleware

feat: connect API client to Supabase
session

feat: migrate application data access to
Supabase

refactor: remove Replit dependencies

chore: prepare Hostinger deployment

fix: secure authenticated API routes

Ne jamais commit :

text



.env

.env.local

SUPABASE_SERVICE_ROLE_KEY

private keys

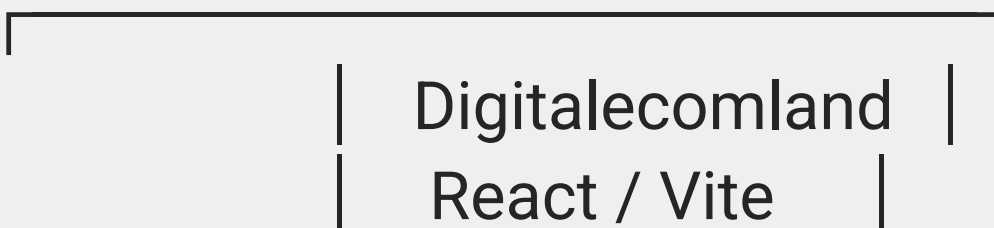
credentials

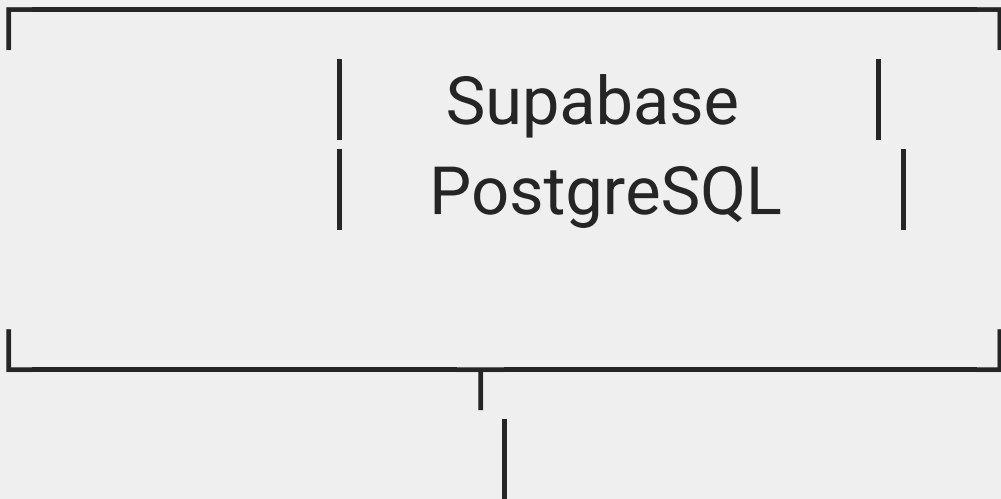
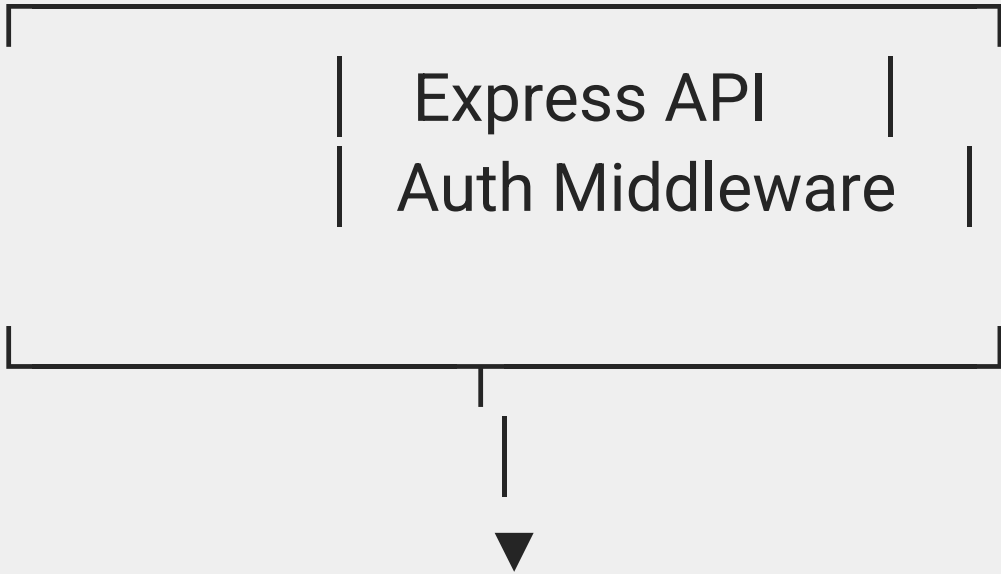
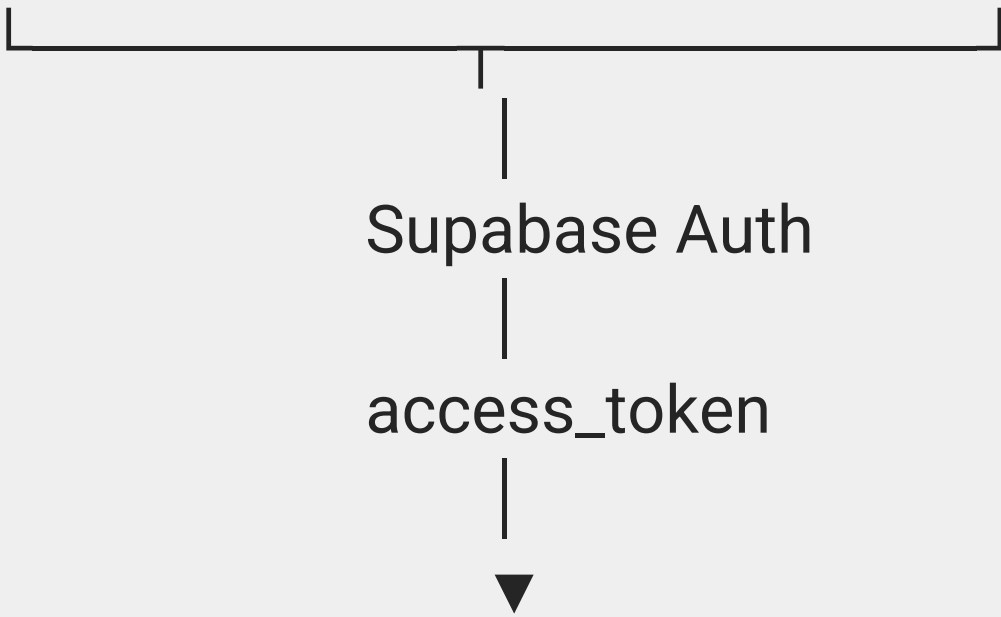
tokens

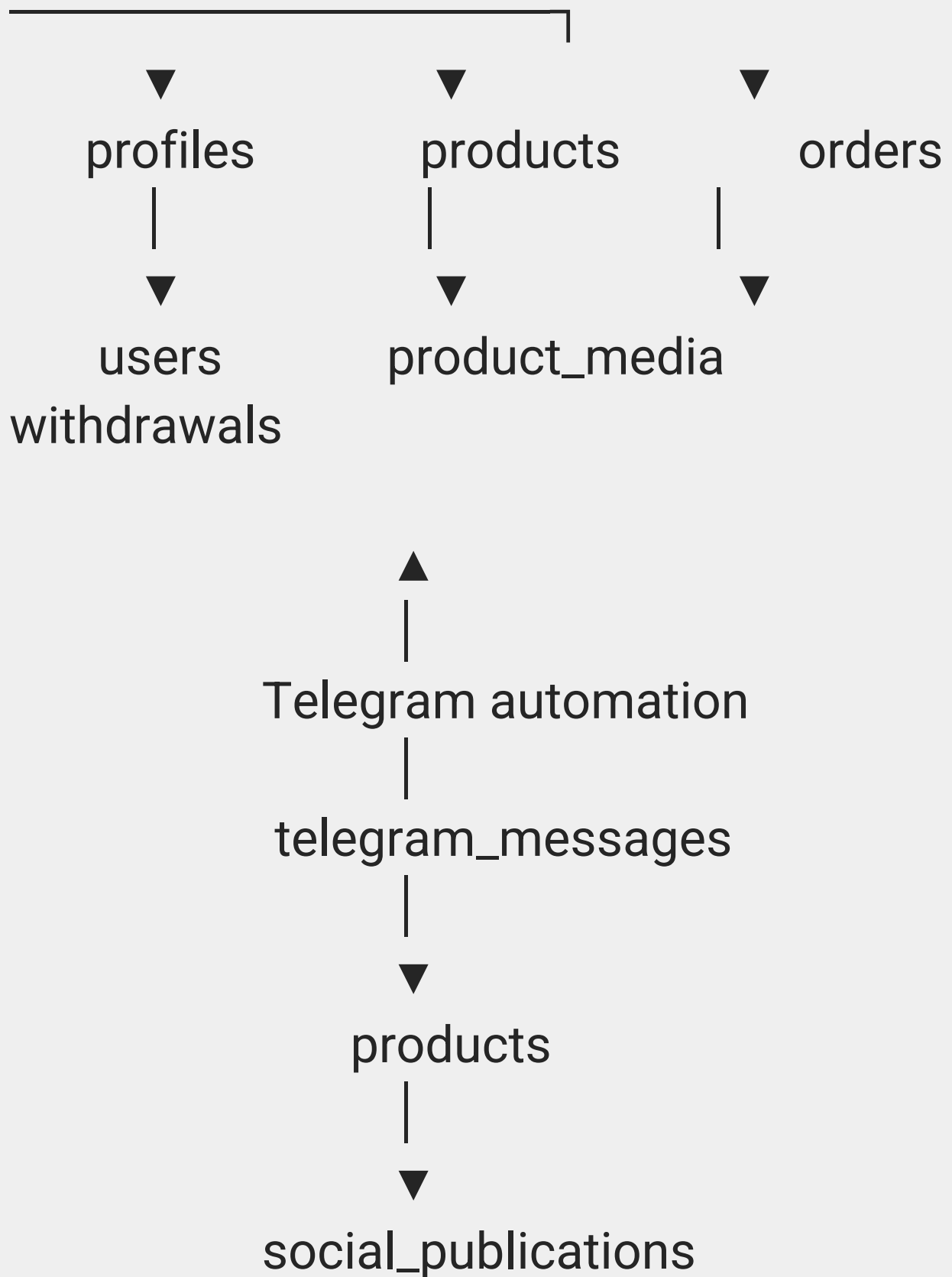
48. Résultat final attendu

À la fin, Digitalecomland doit fonctionner ainsi :

text







Le résultat doit être une application réellement connectée à Supabase, authentifiée, sécurisée, compilable et prête à être déployée sur

Hostinger.

49. Instruction finale à l'agent

Ne prétends pas qu'une modification est terminée si elle n'a pas réellement été appliquée. Après chaque modification, indique :

1. les fichiers modifiés ;
2. ce qui a changé ;
3. pourquoi ;
4. les migrations éventuellement créées ;
5. les tests effectués ;
6. le résultat du build.

Si une modification est impossible à cause des permissions GitHub, arrête la tentative d'écriture et explique précisément le problème au lieu de

simuler un commit.

Commence maintenant par l'audit complet du dépôt solver492/Digitalecomland, puis applique les modifications nécessaires dans l'ordre indiqué ci-dessus.